PrepLang: a data-aware compiler for data-preparation pipelines that reuses its proofs on new batches

Nikhil V

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India

Abstract

A data-preparation pipeline is compiled when its input already exists, and then run again on later batches the compiler never saw. PrepLang, a compiler for a small language of table and image pipelines written in C++17, uses both facts. It reads the input at compile time (every CSV cell, Parquet footers, image headers), proves or refutes the pipeline's data contracts before any data is processed, and deletes every check and operation that this input makes redundant. Every removal stays in the generated pandas and OpenCV program behind a guard that holds the facts about the input it depends on. The program runs unchanged on new batches, runs a removal again only on a batch that breaks those facts, and then gives the result a fresh compilation would give. On 426 later months of five public datasets, the compiled pipelines matched a fresh compilation on 100% of them and never gave a different result. The repository holds 306 test checks with sanitizer, coverage and fuzzing jobs, Lean 4 proofs for a core of the analysis, and one script that reproduces every number in this article.

Keywords: domain-specific language, data-aware compilation, abstract interpretation, speculative optimization, deoptimization, data validation

1. Motivation and significance

Data dependencies and glue code are a lasting source of hidden cost in machine-learning systems [1]. Preparation code is typically written as a pandas [2] or OpenCV [3] script, which learns only by executing whether the column it scales is numeric or whether the pixels it writes to PNG are floats. Validators such as TFDV [4], Deequ [5], Great Expectations [6] and

Email address: atnikhil06@gmail.com (Nikhil V)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.1.0
C2	Permanent link to code/repository used for this code version	`https://github.com/Nikhi00718/preLang/tree/v2.1.0`
C3	Permanent link to Reproducible Capsule	Docker image `ghcr.io/nikhi00718/preplang:2.1.0` (`Dockerfile` in the repository); `eval/reproduce.sh check` builds, tests and regenerates every number of this article from the committed results
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	C++17 (no external libraries); Python 3 with pandas, NumPy, PyArrow, OpenCV, matplotlib; Lean 4; CMake
C7	Compilation requirements, operating environments and dependencies	CMake 3.16+, a C++17 compiler (tested in CI: Visual Studio 2022 on Windows, GCC on Ubuntu and macOS); Python 3.12+ with pandas (2.0+, 3.0 tested), PyArrow, OpenCV, matplotlib (charts); Lean 4.34.1 (optional, for the proofs)
C8	If available, link to developer documentation/manual	`README.md` (user manual), `LANGUAGE.md` (reference), `ARCHITECTURE.md`, `CONTRIBUTING.md`; the READMEs of `eval/`, `paper/`, `proofs/`, `docs/`
C9	Support email for questions	atnikhil06@gmail.com

Table 1: Code metadata (mandatory)

pandera [7] check the data, but apart from the code that transforms it. In batch preparation the input already exists when the pipeline is compiled, and PrepLang reads it. F# type providers read a sample to obtain types [8]; PrepLang obtains values instead, from every CSV cell, Parquet footer and image header. It propagates what it learns (column types, missing values, numeric ranges, image sizes and channels) from statement to statement, settles each contract as true or false before the pipeline touches the data, and deletes the checks and operations that the input makes redundant.

Such proofs are about one batch, whereas the same pipeline later receives files that keep the columns but change the contents. For every removal PrepLang therefore computes the facts about the batch that the removal depends on (its *item certificate*), and the generated program checks them on each new batch. A removal whose facts hold stays removed. A removal whose facts the batch breaks runs again, as code does after deoptimization in dynamic compilers [9, 10]; this article calls it *selective deoptimization*. A numeric column may switch between whole and decimal numbers from one batch to the next when the program compiled with its other type is the same generated program; the program then reads the column as the batch has it. Only when a fact cannot be re-established by run-time code, as with a column that turned from numbers into text, is the whole batch turned away. A user writes a `.prep` program, compiles it against the current batch, reads the diagnostics and deploys the generated program for every later batch.

Table 2 compares PrepLang with the closest tools. Validators know the

Table 2: PrepLang and the closest tools. *Values*: missing values and value ranges.

Tool	Decides at	Knows about the data	Follows code	On a new batch
TFDV [4], Deequ [5], GX [6], pandera [7]	run time	values, from each batch	no	validates it again
F# type providers [8]	compile time	types, from a sample	no	–
Zhuang and Lu [11]	compile time	column types, from the code	yes	–
Gerstl et al. [12]	compile time	R data-frame shapes, optionally from the data	yes	–
Mirji [13]	compile time	declared schemas	yes	checks the output schema
PrepLang	**compile time**	**types and values of every cell; image headers**	**yes**	**admits by certificate; reruns only the removals it breaks**

values but check them apart from the code and again on every batch; compile-time analyses follow the code but know only declared or inferred structure. We found no other tool that decides value contracts at program points from the real data and carries those decisions to later batches.

2. Software description

2.1. Software architecture

The compiler (`src/`, about 11,800 lines of C++17 without external libraries) runs lexer, LL(1) parser, semantic analysis, IR, optimizer, certifier and code generator in turn (Fig. 1). The *schema* readers scan CSV files in parallel and read Parquet footers and PNG and JPEG headers. The *semantic analyzer* computes the abstract dataset state (types, missing values, value intervals, image properties) and decides every contract. The *optimizer* removes, speculatively, what the state shows to have no effect. The *certifier* reruns the analysis under weakened input facts, without reading data, to find the admission certificate and one item certificate per removal. The *back end* generates one Python program that admits a batch and guards every removed instruction with `deopt(i)`, and writes the certificates to `preplang_contract.json`.

The repository also holds `proofs/` (1477 lines of Lean 4 [14]). They prove two things: for any sound and monotone analysis, the generated program

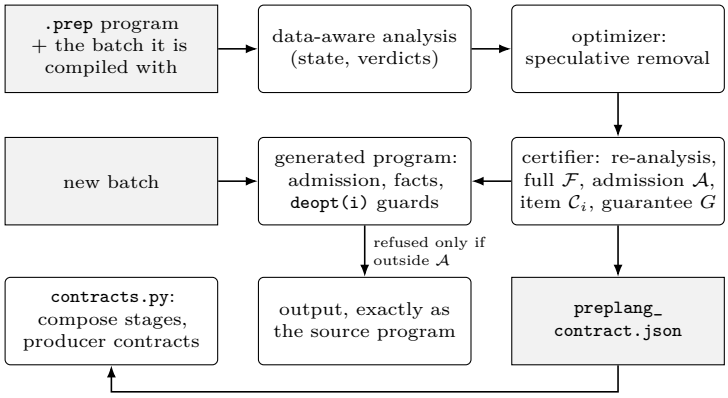


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate $\mathcal{C}_i$.

gives the output of a fresh compilation on every batch it admits; and the analysis of a core of the language is sound and monotone. Joins, groups, dates and images are outside that core. Joins and groups are checked by the differential test of Section 4, dates and images only by unit tests and the examples. `tests/` holds 306 checks in 10 suites.

PrepLang installs with `pip` or runs from the `Dockerfile`; each release also ships compilers for Windows, Linux and macOS. A VS Code extension (`editors/`) shows the diagnostics on save. Continuous integration builds and tests the compiler on Windows (MSVC), Ubuntu and macOS (GCC), compiles every example, runs the tests under AddressSanitizer and UndefinedBehaviorSanitizer with a coverage gate, fuzzes the three readers of untrusted bytes (`.prep`, CSV, Parquet footers) with libFuzzer, and checks the Lean proofs. `LANGUAGE.md` is the reference for the syntax, the options and the output program; `ARCHITECTURE.md` maps each phase to its files.

2.2. Software functionalities

- **Compile-time checking.** Misspelled or removed columns, type errors and broken contracts (`NOT_NULL`, `BETWEEN`, `UNIQUE`, `REFERENCES`, schemas, image channels and sizes, match rates of images and labels) and joins on keys of other types are reported with the line, the offending row and a hint. A lookup join requires each key once; `JOIN ... MANY` takes every match. Groups (`AGGREGATE`, with eight functions from `COUNT` to `STD`) keep their facts.
- **Data-driven optimization.** Proven checks and operations without effect are removed, unused columns are not loaded, and image pipelines

are streamed. Rewrites that change the output, such as merging two `RESIZE`s, are applied only with `--approx`.

- **Tools around the compiler.** `tools/contracts.py` checks one compiled stage's guarantees against the next stage's certificates; `frontend/pandas2prep.py` translates the data-cleaning statements of a pandas script into PrepLang, stopping at the first one it cannot express; and with `--facts` a program compiles from differentially private facts that its data owner released (`tools/release_facts.py`) without opening an input file; the generated program checks those facts on the real data, so every run stays exact.
- **Inspection.** Every phase can be printed (`--tokens` to `--python`), with a plan (`--explain`) and JSON diagnostics (`--json`). `PLOT` writes histograms, bar, scatter and line charts with matplotlib, with contract bounds drawn in, and `REPORT` a web page that sets each column of a run beside what the compiler knew about it.

3. Illustrative example

```
1 LOAD "students.csv" AS students;
2 EXPECT students.age NUMBER;
3 EXPECT students.marks BETWEEN 0 AND 100;
4 EXPECT students.name NOT_NULL;
5 FILTER students WHERE marks >= 50;
6 EXPECT students.marks BETWEEN 60 AND 100;
7 SAVE students AS "clean.csv";
```

Compiled against an eight-row `students.csv` (from `examples/tabular/`), PrepLang proves the first three contracts. It keeps the fourth as a run-time check, since after the filter it knows only that the marks lie in $[50, 95]$. It removes the range and not-null checks speculatively, and `--certificate` shows the facts the range check needs: marks at least 0 and at most 100. Run on a later batch in which a mark is 104, the generated program re-enables the check and stops where a fresh compilation would:

```
Deoptimized line 3: EXPECT students.marks BETWEEN 0 AND 100:
  marks goes up to 104, above 100
ContractError: line 3: students.marks has values outside 0 to 100
```

A batch within the facts runs with both checks removed. A batch whose `age` holds text is refused, because generated code cannot replace an analysis of the column under another type.

The same analysis covers images, from their headers alone. In `examples/images/linked.prep`, images and their labels are linked and checked, resized, normalized to $[0, 1]$ and saved as NumPy arrays. Changing the format to PNG gives, before any pixel is decoded:

```
error[type]: cannot save FLOAT32 pixels as PNG:
             NORMALIZE at line 10 scaled them to [0, 1]
   --> examples/images/linked.prep:13:1
13 | SAVE_IMAGES imgs TO "processed/ml_ready/";
   = hint: image files store 8-bit pixels, so these would come out
     almost black; add CONVERT imgs TO NPY; to keep the float
     values
```

The equivalent hand-written OpenCV script runs without an error and writes almost black images.

4. Impact

PrepLang is a research prototype by one author, begun in a compiler-design course, and has no users outside the author's experiments yet. Its evidence is the evaluation in `eval/`; `eval/evaluate.py` writes every number below from the raw results.

- **Correctness.** 8,000 random programs over four seeds (with lookup and `MANY` joins, the latter in 1,018, and groups), compiled with and without the optimizer, produce the same table, value for value, as an independent Python interpreter of the language that shares no code with the compiler; reused on a second random batch, none of 1,569 and 4,814 runs differs from it. In 1,911 of those batches one numeric column had the other numeric type: 1,341 ran or stopped as the interpreter does, 570 were refused (57 of them cannot be compiled at all), and none was wrong. The test found two bugs (column pruning, a pandas 3 incompatibility) that had made 109 optimized runs wrong, and a third with more seeds (a statistic fill removed as a dead store, 1 wrong run); the CSV fuzzer found an out-of-bounds read on a NUL byte. Each fix has a regression test.
- **Performance.** On a table of 200,000 rows the optimized program runs in 0.81 s instead of 1.32 s after Python start-up, but most real inputs are small: over the 156 programs of the census below that the optimizer changes, the median speedup is 1.05× (at most 1.6×). An image pipeline runs as fast as a hand-written streaming script, in 10 MiB instead of the 922 MiB of a script that loads every image.
- **Optimization.** Across 452 programs, namely the examples plus the programs lifted from GitHub scripts, the optimizer removes 14% of the instructions (2,157 to 1,854), 43 to 8 run-time checks and 2% of the columns read, and changes 156 programs; all 452 write the same files with and without it. The lifted scripts contain no contracts, so

all removed checks come from the examples and the monthly pipelines below.

- **Real programs.** Among 44 preparation bugs that users filed as GitHub issues, PrepLang flags 18 at compile time (6 of them only after the requirement is written as an `EXPECT`). The lifter translates 378 of the 733 public scripts written for the Kaggle Titanic and House Prices data; their programs reproduce the script on 4131 of 4158 new batches, stop or refuse the others, and are never wrong. Of 945 traffic and bike scripts it translates 85, which reproduce the script on all 3710 later real months. For 469 of the 945 the lifter cannot find which file the script reads (a path known only when it runs, or read options that change the table). The lifter does not translate `!=` or `~`: pandas keeps a row with a missing value for them, PrepLang drops it.
- **Comparison.** Five monthly pipelines on public data (UCI traffic, bikes and Beijing air quality; NYC green taxi; Citi Bike Jersey City), each compiled on one month and run on the later ones, match a fresh compilation on 426 of 426 months (100%) and never produce a different result, including the taxi and air-quality months whose columns switched between whole and decimal numbers, which version 2.0 refused. The Citi Bike and air-quality pipelines, with bounds taken from the domain (the New York area, physical ranges), were written after the tool. Each validator decides whether the pipeline may run on a month after learning from the compiled month as its documentation suggests (Great Expectations profiles the columns, pandera and TFDV infer a schema), and is scored against the same fresh compilation. Under that setup, Great Expectations and pandera turn away 96% of all months, and TFDV admits 10 months on which the program then differs. The validators were not built for this decision; the numbers show what it costs to use them for it. All 24 mistakes injected into 5 example programs are reported at compile time, where a script-style pandas program crashes on 20 and writes wrong output on 4.
- **Private compilation.** With $\varepsilon = 1$, compiling from released facts retains all removals in the taxi and traffic pipelines yet only 14% in the bikes pipeline (AIM [15]: 54%), and 74 of 184 Titanic compilations re-enable a removal on their own input; no output of 64,203 runs differs from a fresh compilation.
- **Overhead.** Compiling takes a median of 125 ms, most of it the extra compilations that decide which numeric columns may change type, and at most 24 MiB (65 MiB for a million CSV rows). Admission of a new batch by its certificate alters the run time by -1% to $+2\%$, within

measurement noise.

`eval/reproduce.sh check` (about ten minutes, no network, also in Docker) rebuilds the compiler, runs every test, regenerates every number above from the committed results, and reruns small versions of the differential test and the census. `full` (several hours) downloads the inputs, including the GitHub scripts frozen by URL, commit and hash, and reruns every experiment; `baselines` reruns the validators in pinned environments.

5. Limitations and conclusions

Joins match on one key column, and only Python is generated. A numeric column may change type only when no statement combines it with another column; other type changes are refused. The mechanized proofs cover a core of the analysis, without joins, groups, dates and images. Planned work is joins on several columns and generated code for other targets.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Use of generative AI

Generative AI tools assisted with implementation, literature search, evaluation scripts, analysis and drafting of this manuscript. The author reviewed all text, code and results and takes responsibility for them.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[3] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[6] Great Expectations, "Great Expectations: data quality testing," `https://greatexpectations.io`.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124.

[8] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490.

[9] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43.

[10] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021.

[11] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022.

[12] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[13] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[14] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635.

[15] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.